Polarization in SEDust: Implementation Notes

Abstract

These are maintenance notes for the polarization capability added to SEDust: where the polarized optics come from, how they are assembled, which implementation choices were made and why, and how far the numbers have been checked. The reader is assumed to be someone extending or repairing this code, not someone calling it. The calling interface is documented in `SEDust_user_manual.pdf`; the underlying physics is documented in `aligned_grain_polarization.pdf`. Neither is repeated here.

Contents

1. Scope, and what lives elsewhere

Three documents cover this work and they do not overlap (Table 1). This one answers only the question *how was this built and why that way*. Where physics is needed to justify a coding choice, the relevant section of the theory report is cited rather than restated.

Question	Document
Why polarization at all; alignment theory; the polarized transfer equation; sign and index conventions; validation targets	`aligned_grain_polarization.pdf` (§1–§5, §9–§11)
How an RT code calls the library: signatures, units, what is already integrated	`SEDust_user_manual.pdf`, §5
How the code is built, why each choice was made, what was measured	this note

Table 1: Division of the polarization documentation.

Where this is going. The polarization capability is not an end in itself. The destination is the polarized radiative transfer of Seon (2018): dust scattering plus dichroic extinction by aligned grains, computed with grain optics derived from first principles, in the Monte Carlo formalism of Peest et al. (2017) as extended to aligned spheroids by Peest et al. (2023). Under that contract MoCafe supplies the alignment assumptions (the $f_{\rm align}$ profile and the local magnetic-field geometry) and SEDust returns every material quantity in matrix form: the 4×4 extinction matrix $K(\theta_i)$ with dichroism and birefringence, the phase matrix $Z_{\rm al}(\theta_i; \theta_s, \phi)$ of the aligned population, the random-orientation remainder, and the scattering cross sections. Dichroic extinction, polarized emission, birefringence, and the orientation-resolved cross-section table cover the extinction and emission side of that contract; the scattering side — the fixed-orientation Mueller matrix, the extinction matrix built from forward amplitudes, and their library interface — is now built and verified, and is documented in §8.. What remains is MoCafe's consumption: the frame rotations, direction sampling, peel-off, and the $\exp(-K\tau)$ transfer step, none of which is a SEDust responsibility.

2. The data source

Everything polarized derives from one file, the orientation-resolved Draine & Hensley (2021) astrodust spheroid table:

```
data/dielectric/q_DH21Ad_P0.20_Fe0.00_1.400.dat.gz
```

(porosity $P = 0.20$, iron fraction $f_{\rm Fe} = 0$, axial ratio $b/a = 1.400$). Its layout is not self-describing, so it is recorded here (Table 2).

Property	Value
header	12 lines, skipped
payload order	`((Q(jw,jr,jori), jw=0,1128), jr=0,168), jori=1,3`
repeated	three times: $Q_{\rm ext}$, then $Q_{\rm abs}$, then $Q_{\rm sca}$
one record on disk	the 169 sizes of one (*jw*, *jori*) pair
records read	3 quantities $\times$ 3 orientations $\times$ 1129
total values	$3 \times 3 \times 1129 \times 169 = 1{,}717{,}209$
wavelength axis	`data/dielectric/DH21_wave`, 1129 nodes
size axis	`data/dielectric/DH21_aeff`, 169 nodes
axis file format	two title lines, then the values free-format

Table 2: Layout of the orientation-resolved Q table. The wavelength and size axes are *not* parsed out of the header; they come from the companion grid files, which list the nodes the table was computed on.

Orientation index. The convention is the trap most likely to bite a future reader, and it is worth stating in full (Table 3); see also §3.6 of the theory report.

jori	geometry	meaning
1	$\mathbf{k} \parallel \hat{a}$	propagation along the symmetry axis
2	$\mathbf{k} \perp \hat{a}$, $\mathbf{E} \parallel \hat{a}$	field along the symmetry axis
3	$\mathbf{k} \perp \hat{a}$, $\mathbf{E} \perp \hat{a}$	field across the symmetry axis

Table 3: Orientation index of the DH21 table. $\hat{a}$ is the spheroid symmetry axis.

3. Derived quantities

For a grain whose symmetry axis lies in the plane of the sky and which is perfectly aligned, the polarization cross section is the difference the grain presents to the two linear polarizations, halved:

$$Q_{\rm pol} = \tfrac{1}{2}\,[\,Q(\texttt{jori=3}) - Q(\texttt{jori=2})\,], \tag{1}$$

formed separately from $Q_{\rm ext}$ and from $Q_{\rm abs}$, giving $Q_{\rm pol,ext}$ and $Q_{\rm pol,abs}$. The random-orientation average of the same table is the trace,

$$Q_{\rm ran} = \tfrac{1}{3}\,[\,Q_1 + Q_2 + Q_3\,], \tag{2}$$

which is the modified picket fence approximation (MPFA; §3.2 of the theory report). Cross sections follow from

$$C = Q\,\pi a_{\rm eff}^2, \tag{3}$$

with $a_{\rm eff}$ converted to cm. The alignment efficiency is the Hensley & Draine (2023) Table 1 fit,

$$f_{\rm align}(a) = \frac{f_{\rm max}}{1 + (a_{\rm align}/a)^{\alpha}}, \qquad a_{\rm align} = 0.0749\,\mu{\rm m},\ \ \alpha = 1.80,\ \ f_{\rm max} = 1.00. \tag{4}$$

4. Module map

Table 4 lists what each file is responsible for. Nothing else in the tree touches polarization.

5. Implementation decisions

This is the part worth reading again in six months.

5.1 The accumulation sites, and the two that hide

`sed_grain_loop` branches on `stoch_method` into four cases, and the polarized channel has to be accumulated wherever the total is. There are eight such statements (Table 5). Missing any one of them produces a polarized spectrum that is silently too low rather than an error.

The lesson. The `'draine'` and `'qm'` branches are OpenMP-parallel and accumulate into a thread-local `Jout_local` that is reduced at the end. The `'heuristic'` branch is serial and accumulates into `Jout` directly. A survey of the accumulation sites conducted by searching for `Jout_local` therefore returns only the parallel branches, and an initial

pass over this code identified four sites on exactly that basis. Since `'heuristic'` is the *default* solver, that omission would have made the polarized emission come back identically zero in the default configuration while every other solver looked correct. Any future change that adds a term to the emission sum must be checked against Table 5 branch by branch, not by pattern search.

5.2 Why the `'qm'` rescaling is exact

`qm_solve_grain` does not return a temperature distribution; it returns the emitted spectrum with $C_{\rm abs}$ already folded in. The polarized counterpart is therefore obtained by rescaling:

```
if (Cabs_pop(ii, ir) > 0.0_wp) then
   Jpol_local(ii) = Jpol_local(ii) + &
      wpol * emission_qm(ii) * (Cpol_pop(ii, ir) / Cabs_pop(ii, ir)) / &
      (4.0_wp * PI * lam(ii) * 1.0e-3_wp)
end if
```

This is not an approximation. The temperature distribution $P(T)$ is set by the *heating*, which is governed by $C_{\rm abs}$ and is unchanged by the question of which polarization is being read out. Only the emission readout changes, so

$$C_{\rm pol}\int P(T)B_\lambda(T)\,\mathrm{d}T = \frac{C_{\rm pol}}{C_{\rm abs}}\,C_{\rm abs}\int P(T)B_\lambda(T)\,\mathrm{d}T = \frac{C_{\rm pol}}{C_{\rm abs}}\,\texttt{emission\_qm} \tag{5}$$

holds identically, wavelength by wavelength. Wavelengths with zero $C_{\rm abs}$ emit nothing and are skipped rather than divided by zero.

5.3 The $P(T)$ solver was not touched

No change was made to `calc_P`, `narrow_iterative`, or `qm_solve_grain`. This is deliberate and it is what makes stochastic heating automatically correct for polarization: at every site the same temperature weight multiplies both accumulators, one with $\mathrm{d}n\,C_{\rm abs}$ and the other with $\mathrm{d}n\,f_{\rm align}\,C_{\rm pol}$. The polarized spectrum is thus the same average over the same $P(T)$, and no separate stochastic treatment of the polarized channel is needed (see §5.7 of the theory report for why this is the physically right structure).

5.4 Mixed orientation-average conventions

This is the one open physical wart, and it is documented rather than hidden.

$C_{\rm abs}$ in SEDust comes from our own T-matrix run, whose random-orientation average is computed exactly. $C_{\rm pol}$ comes from the release table, whose available average is the $1/3$ trace, Eq. (2), i.e. the MPFA. The two are different approximations to the same quantity, and the code mixes them: `build_Cpol` adds the polarized channel and deliberately does *not* recompute $C_{\rm abs}$.

Table 6 gives the measured disagreement, $|Q_{\rm abs}^{\rm trace}/Q_{\rm abs}^{\rm exact}-1|$, on the shared $(\lambda, a_{\rm eff})$ grid.

Reading the far-infrared maximum. The 3.46% worst case occurs at $a = 4.2\,\mu$m, a size carrying $\mathrm{d}n/n_{\mathrm{H}} \sim 2.8 \times 10^{-36}$, so it is numerically irrelevant. Restricting to $a \leq 0.5\,\mu$m, which covers 99.999% of the geometric cross section, the worst case drops to 1.98%. At $a \ll \lambda$ both averages approach the same Rayleigh limit, which is why the far infrared is well behaved.

Verdict: deliberately deferred in this release. The mixture is left in place for v1.20, as a decision rather than an oversight. The reasoning is recorded here so that it is not rediscovered later. The mixture is harmless for polarized *emission*, which is a far-infrared phenomenon. It is **not** defensible for polarized *extinction* in the ultraviolet or the optical, and column 8 spans the whole wavelength grid. The `Cpol_ext` output of `dust_extinction` is the same quantity by the same route and carries the same caveat unchanged. Anyone using either shortward of the near infrared must resolve this first. The clean resolution is to recompute C_{abs} on the MPFA convention so that both channels share one average. That changes the existing SEDs, so it breaks their byte-identity with the current reference outputs and has to be done as a separate, deliberately validated step rather than folded into this one. The size of the change is not an argument for leaving it: the correctness argument is that two channels of one calculation should not use two different orientation averages.

What was decided for this release is only *when*, not *whether*. The planned use of these optics — redoing Seon (2018) in the V band, with one or two further optical bands possible — lies entirely in the range where the disagreement is 0.080% or below, while unifying the convention would change C_{abs} and break the byte-identical regression baseline that the rest of v1.20 was validated against. Spending that baseline buys nothing inside the present scope. It has to be spent before any ultraviolet polarized extinction is quoted, and that is the trigger for doing the work.

5.5 f_{align} is the fit, not the release column

`data/release/size_distribution.dat` also ships an alignment column. It agrees with Eq. (4) to a median ratio of 1.0000 but differs by up to 0.32% at the small end. The analytic form is the canonical one in SEDust because it is the published fit, because it is defined on any radius grid rather than only the tabulated one, and because it keeps the alignment model in a single place. Do not mix the two sources: a quantity computed half from one and half from the other is neither.

5.6 Names and surface area

`Cpol` was not renamed to `Cpol_abs`. The name appears at roughly 40 sites and is part of the public surface (`grain_pop_t%Cpol`, the optional `set_pop(..., Cpol_in=...)`). A rename would touch every one of them for no functional gain. The declaration carries an explicit note instead: `Cpol` is the absorption one, `Cpol_ext` its extinction twin.

The polarized extinction was exported as a table column first. A dedicated `dust_cpol_ext` accessor was considered at the time and rejected: `dust_lib` was then an emission-facing API with no extinction surface at all, so such an accessor would have been the module's only extinction entry point, a worse inconsistency than the inconvenience

it removed. The condition attached to that decision was that `Cpol_ext` should join a full extinction API if one ever appeared. It has; see §5.7.

5.7 Why the accessor exists now

The table-only export was right for a library that emitted and did not extinguish. Rebuilding a radiative transfer code on top of SEDust changed the premise: the host would have obtained its emission from a call and its opacity from a parsed text file, and that asymmetry is the whole problem. The file is also written on whatever wavelength grid the driver used, so a host on any other grid has to interpolate optics that the model could have evaluated exactly.

`dust_extinction` closes it. The model object already held everything required, so the accessor adds no physics and no new data source; it performs the same size integral the driver performs, over the populations already built. Two fields were added to `grain_pop_t` to make that possible: `Cpol_ext`, previously a module array only, and `gsca`, the scattering asymmetry, which the emission path never needed and which `sed_init` now interpolates onto the size grid beside $C_{\rm abs}$ and $C_{\rm sca}$, from the same Q table and by the same routine the driver used. Agreement with the table is at the precision the file is written with (a median relative deviation of $\sim 7 \times 10^{-9}$ in each cross section), which is the expected result for one integral evaluated twice.

The table remains, unchanged byte for byte, and remains the right route for a tool that wants the optics without linking the library.

Later revision: the scalar optics went back to the table. The argument above is about the *interface*, and it stands: a host must get its opacity from a call on the same model object that gives it emission, on its own wavelength grid. What changed afterwards is where that call gets its numbers. The scalar transport optics of a dust model do not depend on the transport, so recomputing the size integral inside an RT run buys nothing that reading the recorded integral does not. `dust_extinction` therefore now serves $C_{\rm abs}$, $C_{\rm sca}$, $C_{\rm ext}$ and $\langle\cos\theta\rangle$ from a `data/kext_*.dat` table, attached to the model by the builder (optional `kext_path`; the defaults are the EUV products, the widest grid each model has) and interpolated onto `m%lam`, exactly at the nodes the two grids share. The host's call site is unchanged, and so is the requirement that the file be opened while the working directory is still `sed/` — which is why the read happens in `build_*` and not in `dust_extinction`.

The polarized outputs were deliberately *not* moved. `Cpol_ext` and `Cbir_ext` carry the $f_{\rm align}$ weight, which a host resets cell by cell through `dust_set_alignment`; a column fixed at build time could not follow it. So the routine is now mixed on purpose: scalar optics from a table, polarized optics from the model. The size integral itself is unchanged and is still reachable, for all six outputs, as `size_integrated_extinction` — which is what `calc_kext.x` calls to write the tables, and what the in-tree checks call when they must compare one build of a model against another.

5.8 Compression handling, and a scratch file that leaked

The table ships compressed and Fortran cannot read a deflate stream. Adding a zlib binding was rejected as too heavy for one file. The reader instead shells out once,

```
call execute_command_line('gzip -dc "'//trim(gz_file)//'" > "'// &
```

```
                trim(out_file)//'"', exitstat=estat, cmdstat=cstat)
```

decompresses to a scratch copy, reads it, and deletes it. A fresh clone works with no manual setup step, and neither a 17 MB untracked sibling in `data/dielectric` nor a new build dependency is created. A path not ending in `.gz` is opened directly.

The scratch target: `TMPDIR`, with the process id. The scratch copy is written under `TMPDIR` (else `/tmp`), named with the process id: `q_jori_<pid>.dat` for the polarized Q reader (`q_table_jori`) and `scatmat_aligned_<pid>.dat` for the aligned scattering reader (`scatmat_aligned_mod`); both loaders share the scheme. Naming by process id means concurrent MPI ranks never write the same scratch file, and placing it in `TMPDIR` rather than the launch directory means a read-only launch directory is never written to.

The bug this had. An earlier version wrote a fixed `q_jori_scratch.dat` in the *working* directory. Shell redirection creates the target file *before* `gzip` runs, so when decompression failed the early return left a zero-length `q_jori_scratch.dat` behind there. The failure path now calls `discard_scratch_copy` before bailing out, like every other error exit — and, since the scratch file moved to `TMPDIR/q_jori_<pid>.dat`, a leaked remnant no longer lands in the user's directory at all. The general shape of the bug, an error path that skips the removal the success path performs, is worth watching for in any future edit to this reader: it has eight separate early returns.

6. Verification

6.1 Polarized extinction against the release

The comparison against the release is restricted to $\lambda \geq 0.11\,\mu$m because the reference file turns negative below that. The 0.9422% maximum deviation is not a systematic offset: it sits at $\lambda = 0.112\,\mu$m, right at the sign reversal of the polarized extinction, where the quantity passes through zero and a relative deviation is not a meaningful measure. Away from that point the agreement is at the level of the median.
The 7×10^{-8} figure is the precision the ASCII output format carries, so column 8 and `calc_polext.x` agree as closely as the files allow. They are two independent routes to the same quantity: `calc_polext.x` works straight from `qpol_ext` and the release size distribution, while column 8 goes through `sed_init`, the $\log a$ interpolation onto the SED size grid, and `build_Cpol`.

6.2 Polarized emission

Intrinsic polarization fraction `lamI_pol/lamI_total` under a Mathis ISRF at $U = 1.585$:
The mid-infrared zero is correct rather than a failure: there the emission is dominated by stochastically heated PAHs, which HD23 take to be unaligned.

19.15% versus Planck's 22% is a property of the model, not a bug. Planck measures $p_{\max} = 22.0\,(+3.5, -1.4)\%$ at 353 GHz. The 19.15% here is the *maximum* attainable value, obtained before any geometric factor, and every real line

of sight reduces it, so the model as implemented cannot reach the observed maximum. This traces to the dust model itself: the axial ratio $b/a = 1.4$ is the minimum non-sphericity DH21b found the starlight polarization efficiency to require, and it was not tuned to maximize the submillimeter polarization fraction. Do not close the gap by adjusting $f_{\rm align}$ or the axial ratio without recognizing that as a change to the dust model, requiring its own revalidation against the extinction and emission constraints HD23 fit.

7. First-principles orientation-resolved optics

The table of §2. is Draine & Hensley's precomputed one. SEDust can now regenerate the same orientation-resolved cross sections from the astrodust dielectric function with its own T-matrix engine, so the polarized optics no longer have to be taken on trust from the release file. The random-orientation optics were already SEDust's own (the `q_astrodust` table read by `q_table`); this closes the remaining gap, the orientation-resolved part read by `q_table_jori`.

Recall the three orientations of the incident wave relative to the spheroid symmetry axis $\hat{a}$: jori = 1 is $\mathbf{k} \parallel \hat{a}$; jori = 2 is $\mathbf{k} \perp \hat{a}$ with $\mathbf{E} \parallel \hat{a}$; jori = 3 is $\mathbf{k} \perp \hat{a}$ with $\mathbf{E} \perp \hat{a}$. The polarized and mean combinations are $Q_{\rm pol} = \frac{1}{2}(Q_3 - Q_2)$ and $Q_{\rm ran} = \frac{1}{3}(Q_1 + Q_2 + Q_3)$.

The optics are reached two ways. The *table* is loaded once and interpolated: this is the production path, fast and safe to call from many threads. The *direct* routine `oriented_cross_sections` computes one node from first principles; it is the shared core the table generator loops over, and it also serves an arbitrary (a, λ) or a shape or porosity for which no table exists. It must not be called from inside a loop over radiative-transfer cells: each call is a full T-matrix solve plus an angular integral, and the Mishchenko routines carry their state in `COMMON` blocks and are not thread-safe.

7.1 Rayleigh regime, $x < 0.1$

In the electrostatic limit the oblate spheroid has a diagonal polarizability $\boldsymbol{\alpha} = \mathrm{diag}(\alpha_a, \alpha_b, \alpha_b)$, with α_a for $\mathbf{E}$ along the symmetry axis and α_b for $\mathbf{E}$ transverse, set by the depolarization factors L_a, L_b of the spheroid through $\alpha_i \propto (\varepsilon - 1)/[\,1 + (\varepsilon - 1)L_i\,]$. Absorption and scattering follow as $Q_{\rm abs} \propto \mathrm{Im}\,\alpha$ and $Q_{\rm sca} \propto |\alpha|^2$. The three orientations sample exactly these two polarizabilities: jori = 1 and 3 present the transverse α_b (the field is transverse to $\hat{a}$ in both), jori = 2 presents α_a. So the orientation-resolved cross sections are the two polarization components the random average $(\alpha_a + 2\alpha_b)/3$ is already built from; extracting them before the average is analytic and exact, and Q(jori = 1) equals Q(jori = 3) identically. `rayleigh_limit` returns them through optional arguments, leaving its averaged outputs untouched.

7.2 T-matrix regime, $0.1 \leq x \leq 50$

The T-matrix $\mathbf{T}$ is a property of the particle alone — size, shape, refractive index — independent of orientation and of the incidence and scattering directions. SEDust solves it once for each wavelength and radius with Mishchenko's `tmd_one_scatmat`. Two orientation-resolved quantities are then read off the same $\mathbf{T}$.

Extinction, from the optical theorem. Mishchenko's `AMPL` evaluates the fixed-orientation amplitude matrix $\mathbf{S}(\mathbf{k} \to \mathbf{k}')$ from the stored $\mathbf{T}$ for a particle placed at chosen Euler angles. Taking the scattering direction equal to the incidence

direction (forward) and applying the optical theorem, $C_{\rm ext} = (4\pi/k)\,\mathrm{Im}\,S_{\rm fwd}$ for the incident polarization, gives $C_{\rm ext}$ for each orientation: the symmetry axis is placed along $\mathbf{k}$ (jori = 1) or across it (jori = 2, 3) and the co-polar forward amplitude is read ($S_{\theta\theta}$ for $\mathbf{E}$ along $\hat{\theta}$, $S_{\phi\phi}$ for $\hat{\phi}$).

Scattering, from the phase-matrix integral. The same `AMPL`, swept over all scattering directions at fixed incidence, gives the differential scattering cross section $|\mathbf{S}|^2$; the scattering cross section is its integral over the sphere, $C_{\rm sca} = \int(|S_{\theta\leftarrow p}|^2 + |S_{\phi\leftarrow p}|^2)\,d\Omega$ for incident polarization p, and absorption follows by conservation, $C_{\rm abs} = C_{\rm ext} - C_{\rm sca}$. The fixed-orientation scattered intensity is a trigonometric polynomial of degree $\sim 2N_{\rm max}$ in $\cos\theta$ and in ϕ, so Gauss–Legendre nodes in $\cos\theta$ ($N_{\rm max}+2$ of them) and a uniform azimuth grid ($2N_{\rm max}+2$ points) integrate it to machine precision.

An engine subtlety. Mishchenko's random-orientation expansion `GSP` reuses the `/TMAT/` storage as scratch through an `EQUIVALENCE` and so destroys the T-matrix it reads. The amplitude evaluation needs that same T-matrix afterward, so `tmd_one_scatmat` keeps an intact copy in a second common block and restores `/TMAT/` after `GSP`. Without this the oriented cross sections came out as garbage (negative $C_{\rm ext}$). This corrects a premise stated in the planning note, that the converged T-matrix simply remains available after the solve.

7.3 Geometric-optics regime, $x > 50$

Here the T-matrix does not converge and an asymptotic model is used. Extinction is the extinction paradox, $C_{\rm ext} \to 2A_{\rm proj}$, twice the geometric shadow. For an oblate spheroid of equatorial semi-axis a_s and symmetry semi-axis c ($a_s/c = b/a$), the shadow is πa_s^2 when $\mathbf{k}$ is along the symmetry axis (jori = 1) and $\pi a_s c$ when it is across (jori = 2, 3), giving $Q_{\rm ext}(1) = 2(b/a)^{2/3}$ and $Q_{\rm ext}(2) = Q_{\rm ext}(3) = 2(b/a)^{-1/3}$. The shadow depends on the direction of $\mathbf{k}$ relative to the axis but not on the incident polarization, so extinction carries no polarization at this order.

The polarization in this regime is carried by absorption. In the opaque limit, where a refracted ray is fully absorbed — valid when $\mathrm{Im}(m)\,x \gg 1$ — the absorptivity of an illuminated surface element is one minus its Fresnel power reflectance, and that reflectance is polarization-dependent ($R_s \neq R_p$ away from normal incidence). Integrating over the illuminated half of the spheroid,

$$C_{{\rm abs},p} = \int_{\rm illum} \left[\,1 - \left(|\hat{e}\cdot\hat{s}|^2 R_s(\theta_i) + |\hat{e}\cdot\hat{p}|^2 R_p(\theta_i)\right)\right] dA_{\rm proj},$$

where θ_i is the local angle of incidence and $\hat{s}, \hat{p}$ the local senkrecht and parallel directions. The two grain-frame polarizations ($\hat{e} = \hat{a}$ versus $\hat{e} \perp \hat{a}$) project differently onto the local $\hat{s}, \hat{p}$ basis as they sweep the curved surface, and that is the whole source of the dichroism. A purely geometric ray treatment, with no Fresnel coefficients, would give no polarization at all; the surface reflectance is what makes the regime dichroic, and it is needed to match the release, which carries a real absorption dichroism there — about 3% of the mean, and *growing* with size parameter, the wave-optics signature a ray limit cannot reproduce. This is a deliberate approximation, documented at the code site with its opaque-limit domain of validity; it would matter for a size distribution with large-grain weight, which the astrodust distribution does not have.

7.4 Dispatch, generation, and drop-in

`oriented_cross_sections` selects the regime by x and returns the oriented $(Q_{\rm ext}, Q_{\rm abs}, Q_{\rm sca})$ for one node, with the same branch boundaries and fallbacks as the random-orientation driver. `run_q_jori.x` loops it over the DH21 grid and writes the table in the exact stream the release reader expects (§2.), so the result is a drop-in: `sed_init` and `build_astrodust` already take an optional `qpol_path` that defaults to the release file, and pointing it at the regenerated table switches the polarized optics to SEDust's own with no code change. A full sweep is dominated by the T-matrix nodes (of order 16 hours on one core), so the driver has a wavelength-window `range` mode for process-level parallelism and a `merge` mode that reassembles the windows — a window is not a contiguous slice of the stream, because the wavelength index runs innermost.

7.5 How far it has been checked

Each regime was compared node by node against the release table (Table 9). Across the Rayleigh and T-matrix regimes — every wavelength and radius that carries astrodust weight — the reconstruction matches the release to a median of a few parts in 10^4, in both the mean and the polarized cross sections. The geometric-optics region agrees only coarsely, as an asymptotic model should: the mean absorption to about ten percent and the dichroism to about three-quarters of the release value, both with the correct sign and size-parameter trend. It joins the T-matrix continuously across $x = 50$ in sign and ordering, and it carries negligible weight in the size integral.

End to end, feeding the regenerated table through `calc_polext` reproduces the released polarized extinction to a median of 0.06% over $\lambda \geq 0.11\,\mu$m, against 0.03% for the release table itself; the two agree with each other to a median of 0.02% at the level of the observable. So the few-parts-in-10^4 reconstruction error at the level of a single cross section survives the size integral as a few parts in 10^4 in the polarized extinction, computed entirely from the astrodust dielectric function with no recourse to the release optics.

7.6 Computing named wavelengths

Polarized transfer is run at a handful of wavelengths, not on a whole axis. The `range` mode cannot serve that: its `JW1`/`JW2` are *indices* into a fixed wavelength file, so it reaches only wavelengths that file already carries. Three modes take the physical wavelength instead:

```
./run_q_jori.x lam L1 [L2 ...] [ja=JA1:JA2] [tag=NAME] # wavelengths [um]
./run_q_jori.x lamfile PATH [ja=JA1:JA2] [tag=NAME] # one per line, '#' comments
./run_q_jori.x lammerge STEM FILE [FILE ...] # reassemble ja= windows
```

Each writes a pair, `q_astrodust_jori_P0.20_Fe0.00_1.400.TAG.dat` and its wavelength axis `...TAG.wave`; the default `TAG` is `lamL` for one wavelength and `lamLMIN_LMAX_nN` for several. That pair is what `sed_init` / `build_astrodust` take through `qpol_euv_path` and `qpol_euv_wave_path`. The reader interpolates in $\log\lambda$, so at least *two* wavelengths are needed to fill a band; a one-wavelength file is a valid product to read directly but cannot be interpolated.

Processes, not threads. `ja=JA1:JA2` restricts one invocation to radii `JA1..JA2` of the 169-point size axis so that several *processes* share one wavelength, and `lammerge` puts the column slices back side by side. Threads cannot be used: Mishchenko's solver hands the converged T-matrix to `AMPL` through `COMMON /TMAT/` and keeps its working storage in further `COMMON` blocks (two of them blank), so the core is not re-entrant. Separate processes each get their own copy, and a merged file has been checked byte for byte against a single-process one. Measured: three wavelengths (0.0124, 0.0602 and 0.0912 μm) over 57 processes of three radii each took 9 m 22 s of wall time for 34 m 48 s of processor time. The speedup is 3.7 rather than 57 because the cost is not spread over the radii — a T-matrix solve grows like $N_{\max}^4$ with $N_{\max} \sim x$, so the few radii just below the $x = 60$ ceiling carry nearly all of the work. One radius per invocation (169 processes) lets the operating system interleave them and brings the wall time down to the single most expensive node.

`euv` is an axis switch, not a wavelength. The leading `euv` keyword selects a wavelength *axis file* (`data/dielectric/DH21_wave_euv`), so it cannot be combined with `lam`, `lamfile` or `lammerge`, which carry their own axis. The driver rejects the combination explicitly.

7.7 The polarized band below the Lyman limit

$C_{\rm pol}$, $C_{\rm pol,ext}$ and $C_{\rm bir,ext}$ are read from an orientation-resolved table on the full DH21 wavelength axis, all 1129 nodes from 0.0912 to 39810 μm: by default the release file `data/dielectric/q_DH21Ad_P0.20_Fe0.00_1.400.dat.gz`, or, through `qpol_path`, the regenerated `tmatrix/output/q_astrodust_jori_P0.20_Fe0.00_1.400.dat.gz`, which adds the 4th block and so is the one that makes $C_{\rm bir,ext}$ nonzero. Either way the wavelength coverage is the whole axis. Do not confuse this with the five-band aligned scattering table of §8. — the polarized *extinction* optics are on the full grid.

When a model grid is extended shortward of 0.0912 μm (through `lam_min`), `build_Cpol` looks for the companion table `q_astrodust_jori_euv_P0.20_Fe0.00_1.400.dat.gz`. **That table is not shipped.** The default state is therefore that the extreme-ultraviolet block stays at the zero it was initialized with, and the reader says so on stderr:

```
build_Cpol: no EUV polarized table (...)
          dichroic extinction and birefringence are zero below 9.120E-02 um
          (zero by omission, not by physics).
```

That zero is a documented deficit, not the right value. It is worth being blunt about this, because a zero is easy to mistake for an asymptote. First-principles size integrals (`tmatrix/driver/euv_polarized_optics.f90`) put the alignment-weighted $|C_{\rm pol,ext}|/C_{\rm ext}$ at 1.26×10^{-3} at the 0.0912 μm seam, *rising* to 3.64×10^{-3} near 20.6 eV — a factor 2.9 — before decaying to 1.6×10^{-4} at 100 eV, with the sign *opposite* to the optical band; the reversal falls near 0.106 μm, between 0.1072 and 0.1059 μm. $|C_{\rm bir,ext}|/C_{\rm ext}$ falls monotonically from 7.2×10^{-3} at the seam and changes sign near 51 eV. Zero would be the correct limit only for $x \gg 1$ *and* $|m-1| \ll 1$, and $|m-1|$ is 0.765 at the seam and below 0.1 only above $\sim$50 eV while the grains carrying the signal sit at $x = 5$–50. A sphere has *exactly* zero dichroic extinction and exactly zero birefringence, so the scalar extreme-ultraviolet optics (`q_astrodust_mod`, Mie on the volume-equivalent sphere) could not have supplied these entries either — the shape has to be kept to get them at all.

When the companion table is present. `build_Cpol` reads it and interpolates in $\log\lambda$ and $\log a$, as $C_{\rm abs}$ and $C_{\rm sca}$ are interpolated. A grid running outside the wavelengths the table covers (0.0124–0.0912 μm) is rejected with `status = 9` rather than filled by extrapolation. Generate the table with the `euv` mode of `run_q_jori.x`, or a subset of it with the named-wavelength modes of §7.6.

Why the axis stops at 0.0124 μm **(100 eV).** Above $x = 60$ the only description left is the opaque geometric-optics limit, whose absorption is the Fresnel surface integral described in §7. and which needs the chord optical depth $4\,\mathrm{Im}(m)\,x$ to be large. For DH21 astrodust that quantity is 3.7 at $x = 50$ and 100 eV (2.6% transmission), and it falls below 1 by 200 eV, where the grain is becoming X-ray transparent and neither the extinction paradox nor the opaque Fresnel absorption applies. The floor is set where the physics of every branch used is still valid.

7.8 Certifying the T-matrix above $x = 50$

The extreme ultraviolet pushes size parameters up, so the $x > 50$ cut of §7. would throw away most of the band. In the `euv`, `lam` and `lamfile` modes the T-matrix is therefore attempted up to $x = 60$ (`X_TM_MAX`), under a two-setting certification: every node with $x \in (50, 60]$ is solved twice, at the production (`DDELT`, `NDGS`) = $(10^{-3}, 2)$ and at a coarser-quadrature $(3 \times 10^{-3}, 3)$, and is *kept only* if the two agree to `TOL_CHK` = 5% on every channel. A node that fails falls to geometric optics. Above $x = 60$ geometric optics is used directly. The plain and `range` sweeps keep the older $x > 50$ rule unchanged, so they still reproduce the shipped table byte for byte.
Three details are worth recording.

- **`NDGS` = 4 is not usable** as the second setting: `NGAUSS` = `NDGS` $\times N_{\rm max}$ then exceeds the `NPNG1` = 300 storage limit, so the check would fail for want of array space rather than for want of convergence. `NDGS` = 3 is the largest that leaves the comparison meaningful.

- **`IERR` = 0 is not sufficient at $x \gtrsim 55$.** Mishchenko's own convergence flag can report success and still return a wrong answer: at $\lambda = 0.0912\,\mu$m and $a = 0.9441\,\mu$m ($x = 65.04$) it returns `IERR` = 0 with a value 50% away from its neighbors. Measured at the seam, $x = 51.7$ and 54.7 pass the two-setting check while 57.9, 61.4, 65.0 and 68.9 fail it. The certification exists because the engine's own flag does not carry this information.

- **The polarized channels need an absolute floor.** $Q_{\rm pol}$ is a difference of order-unity numbers, so a purely relative comparison becomes meaningless where it passes through zero. `Q_POL_FLOOR` = 10^{-3} sets the absolute scale below which the polarized channels are not required to agree; after the size integral that corresponds to $|C_{\rm pol}|/C_{\rm ext} \sim 5 \times 10^{-4}$, about 1% of the V-band dichroism.

What the certification costs. Leaving the $x > 60$ nodes at the geometric-optics value, which carries $Q_{\rm pol,ext} = Q_{\rm bir,ext} = 0$ exactly, loses part of the *dichroic extinction*: nothing at the seam, ∼8% of the band total at 0.031 μm, ∼20% at 0.022 μm and ∼46% at 0.0124 μm. The true value lies between the certified figure (a lower bound, 1.06×10^{-4}) and a $1/x$ continuation (an upper bound, 3.14×10^{-4}). No extrapolation is put in place of the missing part; the absolute level is already small there, about 1% of the optical-band signal. $C_{\rm pol}$ — the absorption dichroism, which drives polarized

emission — has no such gap, because the geometric-optics limit obtains it from the opaque-grain Fresnel surface integral.

8. Aligned-grain scattering matrix and the extinction matrix

Section 7. regenerated the orientation-resolved *cross sections*. This section adds the angle-resolved object they left out: the fixed-orientation Mueller matrix $Z(\theta_i;\theta_s,\phi)$ of the aligned astrodust spheroid, and the 4×4 extinction matrix $K(\theta_i)$ that goes with it. Between them they complete the material side of the Peest-formalism contract — SEDust returns every quantity in matrix form; the RT code (MoCafe, whose consumption side is not yet built) does the rotations, sampling, peel-off, and $\exp(-K\tau)$ transfer. The design is set out in `docs/tmatrix_aligned_scattering_plan.md`; this section records how each piece was built and why.

8.1 The engine

`tmatrix/driver/scattering_matrix_oriented.f90` returns the fixed-orientation Mueller matrix Z of the DH21 oblate spheroid ($b/a = 1.4$) for one incidence direction. Two regimes, one output convention.

T-matrix regime, one `AMPL` call. With the T-matrix already converged and left in `COMMON /TMAT/` by `TMD_ONE_SCATMAT` — the same restore of §7. that made the oriented cross sections possible — a single call to Mishchenko's `AMPL` evaluates the 2×2 complex amplitude matrix $\mathbf{S} = [[S_{vv}, S_{vh}], [S_{hv}, S_{hh}]]$ for the chosen incidence and scattering angles. The 16 phase-matrix elements then follow from the standard bilinears in $\mathbf{S}$, copied *verbatim* from Mishchenko's `src/ampld.lp.f` (its `Z11`…`Z44`, Eqs. (13)–(29) of Appl. Opt. **39**, 1026, 2000) so the phase matrix can be checked line by line against the source. Copying rather than rederiving is deliberate: the bilinears carry sign and conjugation conventions that are easy to transcribe wrongly, and the whole point of anchor C below is that this transcription is exact.

Rayleigh regime, analytic dipole. Below $x = 0.1$ the same Z is the electric-dipole matrix, $S_{pq} = k^2\,(\hat{e}_p^{\rm sca}\cdot\boldsymbol{\alpha}\cdot\hat{e}_q^{\rm inc})$ with the spheroid polarizability $\boldsymbol{\alpha} = \mathrm{diag}(\alpha_b,\alpha_b,\alpha_a)$, then the same bilinears. The polarizability comes from `spheroid_dipole_polarizability`, a routine extracted from `rayleigh_limit` (§7.) without changing an operation, so the dipole cross sections that §7. already validated and the dipole matrix here rest on one implementation of α_a, α_b.

The Stokes basis, stated once. Both regimes use Mishchenko's meridional basis of each propagation direction *in the grain frame*, whose z axis is the alignment axis: $\hat{e}_1 = \hat{\theta}$ (call it v), $\hat{e}_2 = \hat{\phi}$ (h), with $Q = I_v - I_h$. This is exactly the basis `AMPL` rotates its amplitudes into, so the analytic dipole and the T-matrix matrix share one convention with no adaptor between them. At $\theta_i = 90^\circ$ the v-polarization is jori = 2 ($\mathbf{E} \parallel \hat{a}$) and h is jori = 3 ($\mathbf{E} \perp \hat{a}$), which is what ties the extinction matrix back to the jori cross sections of §2..

8.2 Symmetries, verified not assumed

Three exact symmetries reduce the stored ranges and, more usefully, are numerical checks on the engine (anchor E). Each was confirmed on a grid of $(\theta_i, \theta_s, \phi)$ before being relied on.

- **Azimuth mirror.** The plane through the axis and the incidence direction is a symmetry plane, so $\phi \to 360^\circ - \phi$ leaves Z unchanged except that the two off-diagonal 2×2 blocks (elements $13, 14, 23, 24, 31, 32, 41, 42$) flip sign. Store $\phi \in [0, 180^\circ]$; residual 3.5×10^{-7}.

- **Equatorial mapping.** The oblate spheroid is symmetric under $z \to -z$, so $\theta_i \to 180^\circ - \theta_i$ maps onto the stored range with $\theta_s \to 180^\circ - \theta_s$, ϕ *unchanged*, and the same off-diagonal-block sign flip,

$$Z(180^\circ - \theta_i;\, 180^\circ - \theta_s,\, \phi) \;=\; S \odot Z(\theta_i;\, \theta_s,\, \phi), \qquad S = \begin{pmatrix} + & + & - & - \\ + & + & - & - \\ - & - & + & + \\ - & - & + & + \end{pmatrix},$$

  the diagonal blocks keeping their sign. Store $\theta_i \in [0, 90^\circ]$; residual 8.8×10^{-16}. Physically this also encodes that the alignment axis is headless.

- **Axisymmetry at $\theta_i = 0$.** There Z must be independent of ϕ up to a frame rotation, $Z(0; \theta_s, \phi) = Z(0; \theta_s, 0)\, R(\phi)$, the $\phi = 0$ matrix being six-element block-diagonal. This is a check, not a storage saving.

The library reconstructs the unstored octants from these relations, applying the ± 1 sign once to the interpolated matrix rather than to eight interpolation corners (§8.5); because each reflection contributes a constant that is the same for every corner of a cell, it factors out of the linear interpolation exactly.

8.3 The generator: one code path

The shared physics lives in `tmatrix/driver/aligned_population_optics.f90`, and both the production driver `run_scatmat_aligned.f90` and the verification program `compare_scatmat_aligned.f90` call it. This is on purpose: the numbers an anchor checks are produced by the exact code the production file is written from, so a passing anchor certifies the shipped path and not a parallel one.

One solve, three products. For each (band, size) in the T-matrix regime a single `TMD_ONE_SCATMAT` solve serves everything: its `GSP` expansion coefficients feed the random-orientation accumulation (the F block below), the `/TMAT/` it leaves valid feeds the `AMPL` sweep over the Z nodes, and its forward amplitudes give the extinction-matrix elements. No quantity is recomputed.

Regime split and the $x > 50$ guard. The size-parameter regimes match `run_scatmat.f90`: $x < 0.1$ analytic dipole, $0.1 \le x \le 50$ T-matrix. Above $x = 50$ no converged oriented matrix exists, so the bin is *omitted from the aligned products* and instead enters $F_{\rm tot}$ as its random-orientation geometric-optics matrix, i.e. scattered as unaligned. Its $C_{\rm sca}$ is tallied, and the driver stops hard if the omitted fraction of the scattering weight exceeds 10^{-6}. It never does for the shipped

bands: the measured omitted fraction is at most 3.1×10^{-15} (at $0.36\,\mu$m) and exactly zero at 0.64 and $0.79\,\mu$m. The omission is thus provably negligible rather than assumed so.

An OpenMP lesson: `-frecursive` for the amplitude routine. The only parallel region is the loop over Z nodes inside one size. `AMPL` and `VIGAMPL` only *read* `COMMON /TMAT/` and otherwise write local automatic arrays, so distinct nodes are independent and each grid slot is written once — which is why a 1-thread and an N-thread run are bitwise identical. The subtlety is in the build: `ampl_oriented.f` is compiled `-frecursive` so its large local arrays become thread-private on the stack, while the serial T-matrix core (`tmd_one.f`, `lpd.f`) keeps its static locals under the base flags. Compiling the amplitude routine with static locals would have shared those arrays across threads and silently corrupted the matrices; the Makefile therefore compiles that one file with the OpenMP-safe flags and the core without them. This is the implementation lesson worth carrying forward: thread-safety of a COMMON-based Mishchenko routine is decided by the compile flags of each file, not by a source edit.

8.4 Products and the η contract

Per band the file (`output/scatmat_aligned_astrodust_P0.20_Fe0.00_1.400.dat`) carries three blocks, all per H.

- K **block**, on the θ_i grid: $C_{\rm ext,al}$, the dichroic $C_{\rm pol,al}$, and the birefringent $C_{\rm bir,al}$ from the forward amplitudes, plus $C_{\rm sca,al}$ as the grid closure $\int Z_{11}\,d\Omega$. Because these come from the forward amplitude at each θ_i, the extinction matrix is *exact at every propagation angle*. This replaces the $\sin^2\psi$ interpolation between the $\psi = 0$ and 90° values that the three-orientation jori table forces, and reduces to that law analytically in the Rayleigh limit (anchor G).
- F **block**: $F_{\rm tot}$ (every grain, identical to `run_scatmat.f90`) and $F_{\rm ref}$ ($f_{\rm align}$-weighted), the six-element random-orientation matrices, α_1-normalized so $\frac{1}{2}\int F_{11}\,d\cos\Theta = 1$ (equivalently $\int F_{11}\,d\Omega = 4\pi$); the header gives $C_{\rm sca,tot}$, $C_{\rm sca,ref}$, $C_{\rm ext,tot}$, $C_{\rm ext,ref}$, from which the absolute differential matrix is $C_{\rm sca}F/(4\pi)$ ($F_{\rm tot}$ with $C_{\rm sca,tot}$, $F_{\rm ref}$ with $C_{\rm sca,ref}$).
- Z **block**: $Z_{\rm al} = \int da\, n(a)\, f(a) Z(a)$, 16 elements, $\mu{\rm m}^2\,{\rm sr}^{-1}$ per H, on $\theta_i = 0(5)90^\circ$ (19) $\times$ $\theta_s = 0(1)180^\circ$ (181) $\times$ $\phi = 0(5)180^\circ$ (37).

The η contract. The file header documents, and the library enforces, the exact rule for varying the alignment degree cell to cell by a single scalar η (the local alignment scale, $f_{\rm cell} = \eta f_{\rm ref}$). The aligned matrix scales linearly, $Z_{\rm al,cell} = \eta Z_{\rm al}$; the extinction matrix likewise, $K_{\rm al,cell} = \eta K_{\rm al}$; the unaligned remainder scattering matrix is $Z_{\rm unal} = [C_{\rm sca,tot}F_{\rm tot} - \eta C_{\rm sca,ref}F_{\rm ref}]/(4\pi)$ in absolute units, whose $\int Z_{11}\,d\Omega = C_{\rm sca,tot} - \eta C_{\rm sca,ref}$; and the unaligned population adds an isotropic $C_{\rm ext,tot} - \eta C_{\rm ext,ref}$ to the extinction (its polarized and birefringent parts vanish). All of this is exact by the linearity of the size integral in $f_{\rm align}$, so three stored integrals give the scattering optics of any η with no re-integration.

8.5 The library: two strictly separated layers

`sed/src/scatmat_aligned.f90` (module `scatmat_aligned_mod`) serves the table to a polarized RT host with the same lifecycle as the rest of the library: initialize once, query along the path.

Initialization (serial, once). `load_scatmat_aligned(path, status)` parses the table into module storage and `free_scatmat_aligned` releases it. The resident cost is dominated by `scm_Z`, whose size is $n_{\theta_i} n_{\theta_s} n_\phi \times 16 \times n_{\rm band}$ doubles: for the production grid ($19 \times 181 \times 37$, five bands) that is about 81 MB, linear in the number of bands, with the F and K arrays adding under 0.1 MB. Queries are trilinear interpolations — a few hundred floating-point operations per scattering event, with no wavelength search once `scatmat_band` has mapped the wavelength to a band. It is wired into `sed_init` / `build_astrodust` through an optional `scatmat_path` argument — exactly the `qpol_path` pattern of §7., except that a `scatmat_path` that fails to load is an error (`status = 3`), because a host that asked for the table depends on it, whereas a missing polarized-optics table only disables polarized emission. This layer is not thread-safe and is called from serial code.

Path queries (pure reads, OpenMP-safe). Six routines write only their own outputs and read the loaded arrays, so they are safe to call concurrently from photon threads:

- `scatmat_band(lambda_um, iband, exact)` — nearest stored band, so the hot path takes an integer and no wavelength search runs per event;
- `extinction_matrix_aligned(iband, theta_i, eta, kmat)` — the 4×4 matrix with diagonal $\eta C_{\rm ext,al}$, $K(1,2) = K(2,1) = \eta C_{\rm pol,al}$, $K(3,4) = \eta C_{\rm bir,al}$, $K(4,3) = -\eta C_{\rm bir,al}$, θ_i folded into $[0, 90°]$ because K is even under the equatorial reflection;
- `mueller_matrix_aligned(iband, theta_i, theta_s, phi, z)` — trilinear interpolation plus the symmetry reconstruction of §8.2, at the reference alignment ($\eta = 1$; the caller scales);
- `mueller_matrix_random(iband, big_theta, f_tot, f_ref)` — the two six-element random-orientation matrices;
- `mueller_matrix_total(iband, theta_i, theta_s, phi, eta, z)` — the recommended host call: the *absolute* combined matrix $z = \eta Z_{\rm al} + L(\pi - \sigma_2) F_{\rm unal}(\Theta) L(-\sigma_1)/(4\pi)$ with $F_{\rm unal} = C_{\rm sca,tot} F_{\rm tot} - \eta C_{\rm sca,ref} F_{\rm ref}$, $\cos\Theta = \cos\theta_i \cos\theta_s + \sin\theta_i \sin\theta_s \cos\phi$ and the meridional rotation angles σ_1, σ_2 from `meridional_scattering_angles` (closed `atan2` forms, poles included), so a host need not reassemble the mixture from the arrays;
- `scattering_cross_sections(iband, theta_i, eta, csca_aligned, csca_unaligned [, csca_pol_aligned])` with `csca_unaligned` = $C_{\rm sca,tot} - \eta C_{\rm sca,ref}$ and the optional `csca_pol_aligned` = $\eta C_{\rm sca,pol,al}(\theta_i) = \eta \int Z_{12}\, d\Omega$, the aligned matrix's polarized scattering cross section (Peest et al. 2023; the random remainder integrates to zero), on the same $[0, 90°]$ fold.

`scm_csca_pol_al(nti,nband)`, the polarized scattering cross section $\int Z_{12}\, d\Omega$ that `csca_pol_aligned` returns, is formed at load time with the generator's own closure quadrature — the same rule that closes $\int Z_{11}\, d\Omega$ to $C_{\rm sca,al}$ — so it needs no extra file column. The loaded grids and arrays are also exposed `public, protected`, so a host that prefers to inline its own lookups can take them once at startup and never call back. The whole cell dependence enters through just two scalars the host already holds: η, and $\theta_i = \arccos(\hat{k} \cdot \hat{B})$ from one dot product.

The alignment guard (status 4). The K and Z arrays were integrated over size once, under the profile recorded in the header; the sanctioned runtime variation is η, not a change of profile. So when a loaded scattering table is present and the host calls `dust_set_alignment` or `dust_set_alignment_profile` with a profile that differs from the recorded one, the setter returns a *new* status code 4. It is non-fatal: the $f_{\rm align}$ update still completes (the emission and $C_{\rm pol,ext}$ channels re-integrate live and honor the new profile), and the code only signals that the loaded scattering matrices no longer correspond to the model's alignment. A different profile requires regenerating the table (`run_scatmat_aligned.x profile=FILE`), which takes minutes. A tabulated profile never matches the recorded analytic one and so always raises the flag.

8.6 Birefringence through `dust_extinction`

`dust_extinction` gained an optional `Cbir_ext` output, the $f_{\rm align}$-weighted size integral of the jori table's 4th-block birefringence,

$$C_{\rm bir,ext}(\lambda) = \sum_a \mathrm{d}n_{\rm Ad}(a)\, C_{\rm bir,ext}(\lambda, a)\, f_{\rm align}(a),$$

mirroring `Cpol_ext` exactly. It is zero when the loaded table carries no 4th block (as the release table does not), so the default path is unchanged, and it is the same birefringence the aligned K block carries at $\theta_i = 90^\circ$, reached here without loading the aligned scattering table. This closes the gap noted in the previous release, that `dust_extinction` did not yet return the birefringence.

8.7 Verification

Every anchor of the plan passes (Table 10). The engine anchors (A–H) run at single sizes through `compare_scatmat_aligned.x` — anchor H adds the orientation-averaged amplitude engine at general geometry ($\theta_i = 40^\circ$, $\theta_s = 70^\circ$, $\phi \in \{30, 120, 250\}^\circ$) against $L(\pi - \sigma_2)F(\Theta)L(-\sigma_1)$, which a wrong σ sign fails at order unity to order 30. The library checks, including the failing-tolerance certification tests [3a]–[3e] of `test_scatmat_aligned.x` — the absolute 4π closure, the $\int Z_{12}$ polarized cross section, the `mueller_matrix_total` closure, and the $\theta_i = 0$ rotation sign — run on the production table.

Cost of the production run. The five UBVRI bands (0.36, 0.44, 0.55, 0.64, 0.79 μm) take about 8 minutes at 32 threads (about 26 minutes at 8); the file is 138 MB, 38 MB gzipped. The existing regressions are unchanged: `calc_polext` reproduces the release to a median of 0.0296% as before, and every `sed`, `mc` and `tmatrix` target builds clean.

8.8 Which wavelengths the table carries

The shipped table holds five bands, 0.36, 0.44, 0.55, 0.64 and 0.79 μm (`BANDS` in `run_scatmat_aligned.f90`), on the grid $\theta_i = 0(5)90^\circ$ (19) $\times$ $\theta_s = 0(1)180^\circ$ (181) $\times$ $\phi = 0(5)180^\circ$ (37). That is the intended scope: polarized transfer is essentially never run on a whole wavelength grid, and five optical bands cover the update this work is for. What matters is that a wavelength one later needs can be computed, not that everything is precomputed now.

`run_scatmat_aligned.x` does already take wavelengths on its command line (`./run_scatmat_aligned.x 0.44 0.55 0.79`), so a different set is one run away. Three limitations of that path should be stated plainly rather than discovered:

- **The output stem is fixed** (`f_stem//'.dat'`), so a custom run *overwrites* the shipped five-band table. Move or rename the shipped file first.
- **There is no merge mode.** A band cannot be added to an existing table; every requested band is recomputed from scratch into a new file (the five together are the 8-minute run above).
- **There is no size-window split and no large-x certification.** The driver has no `ja=` equivalent of §7.6, and it applies only the plain `X_LARGE` = 50 cut, without the two-setting cross-check of §7.8.

None of these is hard to lift; none of them is done, and the wavelengths in hand have not required it.

8.9 Why the library reads tables and does not solve

The library never runs a T-matrix solve at run time. The reason is not the size of the code — the engine is already in the tree and the drivers link it — but that convergence cannot be certified at $x \gtrsim 55$ from inside a transport loop: as §7.8 records, Mishchenko's `IERR` can report success on a node that is 50% wrong, and the check that catches it costs a second solve at different settings. A silently wrong optic is worse than a missing one, and the transport loop has nowhere to report a failure to. The tables are generated offline, where a node can be rejected and the rejection counted in the file header. This is the current policy rather than a permanent decision: the T-matrix core itself is expected to be revised, and a certification cheap enough to run inline would change the trade.

9. Status codes, and the relation to v1.00

9.1 `sed_init` / `build_astrodust` status codes

Both take an optional `status`; 0 is success, and when the argument is absent the readers keep their message-and-stop behavior instead. The v1.20 codes are those of Table 11.

Code 8 is declared for an unreadable extreme-ultraviolet polarized table. Note what the current code actually does with an *absent* companion table: that is not an error — `build_Cpol` reports it on stderr and leaves the band zero (§7.7) — so the status a caller sees in practice is 9, raised when a table that *was* read does not cover the requested grid.

`dust_set_alignment` and `dust_set_alignment_profile` have their own independent numbering: 1, 2, 3 for an out-of-range `a_align`, `alpha_align` or `f_max`, and 4 for the non-fatal alignment guard of §8.5. Do not read a 4 from a setter as the `qpol_path` failure above.

9.2 v1.20 is v1.00 plus polarization

The two maintained branches are a superset relation, not a fork. Everything scalar is identical: the same builders (`build_astrodust`, `build_dl07`, `build_zubko`, `build_from_files`), the same `dust_emission` and

dust_extinction, the same lam_min extreme-ultraviolet extension, and the same calc_kext.x products. What v1.20 adds is the polarized material: Cpol, Cpol_ext, Cbir_ext, $f_{\rm align}$, the aligned scattering table and its library layer. The only incidental difference is the status numbering — what v1.00 calls 3 and 4 (the dielectric function load and the lam_min floor) are 6 and 7 here, because 3, 4 and 5 were taken by the polarized paths. A scalar-only v1.20 run (load_polarized_optics = .false.) returns the scalar Cext, Cabs, Csca, gbar and the total SED bit-identical to the polarized build at zero alignment.

10. Limits and extension paths

These are limits of the published optics, not of the code.

Birefringence is now computed; the release table has none. The release table stores $Q_{\rm ext}$, $Q_{\rm abs}$ and $Q_{\rm sca}$ only, no phase retardation, so from it $C_{\rm circ} = 0$. The first-principles route of §7. already evaluates the fixed-orientation amplitude matrix (Mishchenko's AMPL, in tmatrix/src/ampl_oriented.f) and used only its imaginary forward part, for extinction. Keeping the real part as well costs nothing — it comes from the same amplitude call — and gives the birefringence, the phase retardation that couples U and V. It is stored as an optional 4th block of the regenerated table, $Q_{\rm re}({\rm jori}) = (4\pi/k)\,{\rm Re}[S_{\rm fwd}]/(\pi a^2)$, the real twin of $Q_{\rm ext}$, from which $C_{\rm bir} = \frac{1}{2}(Q_{{\rm re},3} - Q_{{\rm re},2})\,\pi a^2$. The reader exposes it as qbir_ext with a has_bir flag that stays .false. for an older 3-block table, so nothing about the default path changes. No astrodust reference for the birefringence exists, so it is certified internally by Kramers-Kronig: Re and Im of the forward-amplitude difference are a Hilbert pair, and the directly computed birefringence agrees with the Hilbert transform of the already-validated dichroism to a median $\sim 0.1\%$ over the interior of the grid (the transform itself self-tests to 2×10^{-5} on a Lorentz oscillator; the residual is the finite-range truncation). What remains is to return it through dust_extinction and to consume it in the $U \leftrightarrow V$ transfer term, both on the host side.

The aligned scattering matrix is now computed. The release gives the integrated $Q_{\rm sca}$ with no angular information, so scattering by an aligned spheroid could not be modeled from it. It is now modeled from first principles: the fixed-orientation Mueller matrix $Z(\theta_i; \theta_s, \phi)$ of the aligned astrodust spheroid is built from the same fixed-orientation amplitude (ampl_oriented.f) used above for extinction, size-integrated over the astrodust distribution with the alignment weight, and paired with the 4×4 extinction matrix $K(\theta_i)$ from the forward amplitudes. It is written for five optical bands by run_scatmat_aligned.x and served to a polarized RT host through a two-layer library API. §8. is the full account, and §8.8 covers the wavelength coverage and what computing a different band involves.

The random-orientation scattering matrix does exist, in five optical bands. This is a different object from the previous paragraph and should not be confused with it. Dropping the alignment, a randomly oriented spheroid with a plane of symmetry has six independent scattering-matrix elements rather than sixteen, and those *are* computed in the tree: tmatrix/driver/run_scatmat.f90 restores Mishchenko's MATR expansion, accumulates the single-size matrices over the HD23 size distribution with $C_{\rm sca}\,{\rm d}n/n_{\rm H}$ weighting, and writes tmatrix/output/scatmat_astrodust_P0.20_Fe0.00_1.400.dat: one block per wavelength, 181 rows at θ =

$0, 1, \ldots, 180°$, with columns $F_{11}, F_{22}, F_{33}, F_{44}, F_{12}, F_{34}$. It answers what an unaligned grain population scatters; it does not answer the aligned question above.

The stored wavelengths are 0.36, 0.44, 0.55, 0.64 and 0.79 μm, roughly *UBVRI* — the same five bands the aligned table carries, and for the same reason (§8.8). Five bands are the deliverable, not a sweep left half-finished: polarized transfer is run at a few wavelengths, the V band first with one or two further optical bands possible, and the whole 1129-point grid would cost about 1.7 hours spent almost entirely on wavelengths nothing is waiting for. What the design owes is a way to compute a wavelength when one is wanted, and that costs a run rather than a code change: `./run_scatmat.x all` does the full grid, or a list of wavelengths does a subset, at about 5.5 s per wavelength.

Far infrared needs neither. In that regime the albedo is essentially zero, so the transfer requires only the extinction matrix and the emission vector, both fully determined by what is exported. The HAWC+ 154 μm comparison lies inside this regime, so the far-infrared milestone is reachable with the present data and should be attempted before any scattering work.

Astrodust only. DL07 and Zubko have no polarized optics. `build_dl07` releases any polarized arrays left by a previous astrodust initialization, so `lamI_pol` returns zeros rather than stale values.

10.1 Cost of `build_Cpol`

Loading the orientation-resolved table is the most expensive part of model construction (Table 12).

The orientation-resolved arrays `qext_j`, `qabs_j` and `qsca_j` are deallocated as soon as the combinations of Eqs. (1)–(2) have been formed. They are intermediate quantities: nothing downstream of `build_Cpol` reads them, and every later step uses the five derived arrays only. Releasing them removes 13.7 MB, about two-thirds of what the table would otherwise hold, and leaves 7.6 MB resident for the rest of the run. The three arrays are `private` to `q_table_jori`, so no consumer can depend on their surviving the build.

Measured on `main_astrodust.x`, the maximum resident set size falls from 51232 kB to 40088 kB, a reduction of about 10.9 MB. The peak does not fall by the full 13.7 MB because it is a high-water mark: once the orientation-resolved arrays are gone, the later SED solution stages set the peak instead. The steady-state occupancy just after `sed_init` does drop by the full 13.7 MB.

11. Reproducing the numbers

From `SEDust/sed/`:

```
make calc_polext.x # standalone polarized-extinction check
./calc_polext.x # -> output/polarized_extinction_ours.dat
                          # lambda, ours, reference, ratio

make calc_kext.x # full-pipeline extinction table, all dust models
./calc_kext.x astrodust # -> ../data/kext_astrodust_MW.dat, 8 columns
```

```
                        # ./calc_kext.x astrodust euv (EUV grid)
                        # ./calc_kext.x astrodust 0.03 (lam_min [um])
                        # ./calc_kext.x dl07 | zubko | from_files ...

make libsedust.a # library, for the emission path
```

`calc_polext.x` prints the median and maximum deviation against the release directly, restricted to $\lambda \geq 0.11\,\mu$m. Column 8 of the extinction table should reproduce it to the ASCII precision quoted in Table 7; if the two disagree by more than that, the $\log a$ interpolation in `build_Cpol` or the size grid it interpolates onto is the place to look, since that is the only step separating the two routes.

For the emission fractions of Table 8, build a model with `build_astrodust`, evaluate `J_Mathis` at $U = 1.585$, and call `dust_emission` with `lamI_pol` requested; the example in §5.4 of the user manual is the complete recipe.

12. References

- Draine, B. T., & Hensley, B. S. 2021b, ApJ, 919, 65 (DH21b). 2021ApJ...919...65D
- Hensley, B. S., & Draine, B. T. 2023, ApJ, 948, 55 (HD23). 2023ApJ...948...55H
- Peest, C., Camps, P., Stalevski, M., Baes, M., & Siebenmorgen, R. 2017, A&A, 601, A92. 2017A&A...601A..92P
- Peest, C., et al. 2023, A&A, 673, A112. 2023A&A...673A.112P
- Planck Collaboration 2020, A&A, 641, A12. 2020A&A...641A..12P
- Reissl, S., Wolf, S., & Brauer, R. 2016, A&A, 593, A87 (POLARIS I). 2016A&A...593A..87R
- Seon, K.-I. 2018, ApJ, 862, 87. 2018ApJ...862...87S

File / entity	Responsibility
`sed/src/q_table_jori.f90`	The reader. Decompresses, parses the three quantity blocks, validates (monotone axes, finite values, no trailing data), forms Eqs. (1)–(2), and supplies `falign_hd23(a)`, Eq. (4). Publishes `qpol_ext`, `qpol_abs` and `qran_*`; the orientation-resolved `qext_j`/`qabs_j`/`qsca_j` are `private` and are deallocated once the combinations have been formed.
`sed_astrodust.f90`: `build_Cpol`	Interpolates $Q_{\rm pol}$ from the 169-node table axis onto the size-distribution grid in $\log a$, exactly as $C_{\rm abs}$ and $C_{\rm sca}$ are, multiplies by $\pi a_{\rm eff}^2$, and fills `falign_ad`. Checks that the polarized and Q wavelength grids agree node for node.
`sed_astrodust.f90`: `Cpol`, `Cpol_ext`, `falign_ad`	Module-level arrays, `(NLAM,NA)` and `(NA)`. `Cpol` is the *absorption* one and drives polarized emission; `Cpol_ext` is its extinction twin and drives dichroic extinction.
`dust_model_mod.f90`: `grain_pop_t`	Carries `Cpol(:,:)` and `falign(:)`. Both unallocated is how the solver recognizes an unpolarized population. Also carries the extinction-side `Cpol_ext(:,:)` and `gsca(:,:)`, read only by `dust_extinction`.
`sed_astrodust.f90`: `set_pop`	Optional `Cpol_in`, `falign_in`. Both must be supplied together; supplying neither leaves the population unpolarized. The extinction-side `Csca_in`, `Cpol_ext_in` and `gsca_in` are optional and independent; a population that omits them contributes zero to those terms of the size integral.
`sed_astrodust.f90`: `sed_grain_loop`	Optional `Cpol_pop`, `falign_pop`, `Jpol`, again all three or none (`do_pol`). Accumulates the polarized channel alongside `Jout` in every solver branch (§5.1).
`sed_astrodust.f90`: `dust_emission`	Optional `lamI_pol`, last argument. Sums `Jpol` over the populations that carry polarized optics and applies the same unit convention as `lamI_total`.
`sed_astrodust.f90`: `dust_extinction`	The extinction accessor, per H atom on `m%lam`. It integrates $C_{\rm pol,ext}\, f_{\rm align}$ (and the birefringent twin) over the size distribution of every population — these are the same numbers column 8 of the table carries, without the file. $C_{\rm abs}$, $C_{\rm sca}$ and $\langle\cos\theta\rangle$ it later stopped integrating: it reads them from the precomputed `kext` table instead, while the polarized outputs stay computed because their $f_{\rm align}$ weight is runtime state (§5.7). The first-principles integral of all six is `size_integrated_extinction`.
`sed/src/calc_polext.f90`	Standalone check: computes the polarized extinction directly from `qpol_ext` and the release size distribution, and compares against the published `polarized_extinction.dat`. Independent of `sed_init`.
`sed/src/calc_kext.f90`	The extinction-table driver, one executable for every dust model (`./calc_kext.x astrodust | dl07 | zubko | from_files`). In its `astrodust` mode it writes column 8, `C_polext/H`, of `data/kext_astrodust_MW.dat` as $\sum_a \mathrm{d}n_{\rm Ad}(a)\, C_{\rm pol,ext}(\lambda,a)\, f_{\rm align}(a)$. Only that mode writes the column, because only the astrodust builder fills `Cpol_ext`.

Table 4: Responsibility of each file touched by the polarization update.

branch	context	accumulates into	pattern
`'draine'`	equilibrium grain	`Jpol_local`	thread-local
`'draine'`	stochastic, sum over $P(T)$	`Jpol_local`	thread-local
`'heuristic'`	equilibrium grain	`Jpol`	**direct**
`'heuristic'`	stochastic, sum over $P(T)$	`Jpol`	**direct**
`'qm'`	QM solve, rescaled	`Jpol_local`	thread-local
`'qm'`	GD fallback after QM failure	`Jpol_local`	thread-local
`'qm'`	equilibrium grain	`Jpol_local`	thread-local
`'equil'`	every grain	`Jpol`	direct

Table 5: Where the polarized emission is accumulated. The two boldface rows are the ones a search for the `Jout_local` pattern does not find.

band	median	maximum
far infrared ($\lambda > 30\,\mu$m)	0.022%	3.46%
restricted to $a \leq 0.5\,\mu$m		1.98%
mid infrared	0.033%	
near infrared	0.038%	
optical	0.080%	
ultraviolet	0.21%	9.53%

Table 6: Disagreement between the trace average and the exact orientation average of $Q_{\rm abs}$.

Check	Result
Column 8 vs. released `polarized_extinction.dat`	median 0.0296%, max 0.9422%, 992 points
Column 8 vs. `calc_polext.x`	agree to 7×10^{-8} relative
Columns 1–7 of `kext_astrodust_MW.dat`	byte-identical to the pre-change file
`astrodust_irem_ours_S1/S2/PAH.dat`	byte-identical
`dl07_sed_ours_mw31_60.dat`	byte-identical
Build	zero warnings

Table 7: Verification of the polarized extinction path and regression of everything else.

λ	fraction
$12\,\mu$m	0.00%
$100\,\mu$m	13.67%
$154\,\mu$m	17.15%
$350\,\mu$m	18.79%
$850\,\mu$m	19.15%
median over 300–3000 μm	19.18%

Table 8: Intrinsic polarization fraction of the emitted spectrum, before any geometric factor.

Regime	x	Quantity	median $\lvert$ours$/$HD23 $-1\rvert$
Rayleigh	< 0.1	$Q_{\rm ran}$ (abs)	2.3×10^{-4}
		$Q_{\rm pol}$ (abs)	3.7×10^{-4}
T-matrix	0.1–50	$Q_{\rm ran}$ (ext)	2.1×10^{-4}
		$Q_{\rm pol}$ (ext)	8.4×10^{-4}
		$Q_{\rm ran}$ (abs)	1.9×10^{-4}
		$Q_{\rm pol}$ (abs)	7.6×10^{-4}
GO	> 50	$Q_{\rm ran}$ (abs)	9.3×10^{-2}
		$Q_{\rm pol}$ (abs)	3.3×10^{-1}

Table 9: Node-by-node reconstruction against the HD23 release table. The geometric-optics dichroism is at ~ 0.75 of the release with matching sign and trend; its size-parameter region carries no astrodust weight.

	Check	What it certifies	Result
Engine (single size)			
A	closure $\int Z_{11}\, d\Omega$ vs $C_{\rm sca}$(jori)	phase-matrix normalization	2.7×10^{-7}
B	optical theorem vs $C_{\rm ext}$(jori)	forward-amplitude geometry	2×10^{-14}
C	orientation average vs GSP $F(\Theta)$	the bilinears, incl. F_{12}, F_{34} signs	1.4×10^{-6}
D	dipole vs closed form	Rayleigh implementation	2×10^{-16}
	dipole vs T-matrix at $x \sim 0.1$	regime continuity, $\lvert\delta Z\rvert/Z_{11}$	0.5–1.1%
E	ϕ mirror	azimuth symmetry	3.5×10^{-7}
	equatorial mapping	$z \to -z$ symmetry	8.8×10^{-16}
G	Rayleigh $K(\theta_i)$ vs $\sin^2$ law	extinction-matrix angle law (exact)	3.3×10^{-16}
	T-matrix $K(\theta_i)$ vs $\sin^2$ law	at $x = 0.137$	6.3×10^{-4}
H	general geometry	amplitude engine vs $L(\pi - \sigma_2)F(\Theta)L(-\sigma_1)$, θ_i=40°, θ_s=70°; wrong σ sign fails O(1)–O(30)	4.1×10^{-5}
Generator (size-integrated)			
F	K vs independent jori integrals	$C_{\rm ext}$	1.6×10^{-6}
		$C_{\rm pol}$	2.0×10^{-5}
		$C_{\rm bir}$	1.3×10^{-7}
Library (production table)			
	reader round-trip	parse vs re-emit	bitwise 0
	symmetry reconstruction	unstored octants	bitwise 0
	η algebra	linear scaling	residual 0
	$K(90°)$ vs jori vs `dust_extinction`	$C_{\rm pol}$ / $C_{\rm bir}$	1.6×10^{-5} / 8.0×10^{-5}
	`rt_example` closures	end-to-end consumption	0.997–1.001
[3a]	random absolute closure	$(4\pi)^{-1} \int C_{\rm sca,tot} F_{11}\, d\Omega = C_{\rm sca,tot}$	3.5×10^{-4}
[3b]	stored Z_{11} re-integration	vs the table's $C_{\rm sca,al}$ column (ASCII rounding)	3.3×10^{-6}
[3c]	$C_{\rm sca,pol}$ vs $\int Z_{12}\, d\Omega$	routine vs direct; $\lvert C_{\rm sca,pol}\rvert/C_{\rm sca,al} \leq 0.101$	exact
[3d]	`mueller_matrix_total` closure	$\int z_{11} = \eta C_{\rm sca,al} + C_{\rm sca,tot} - \eta C_{\rm sca,ref}$; $\int z_{12} = \eta C_{\rm sca,pol}$; $\eta \in \{0, 0.37, 1\}$	2.6×10^{-4} / 7×10^{-7}
[3e]	rotation sign at $\theta_i = 0$	$Z(0; \theta_s, \phi) = Z(0; \theta_s, 0)\, L(-\phi)$	6.8×10^{-5}

Table 10: Verification of the aligned scattering matrix, its extinction matrix, and the library API. Engine and generator anchors are those of the plan's §5, plus anchor H (the amplitude engine at general geometry) and the failing-tolerance library certification tests [3a]–[3e] of `test_scatmat_aligned.x`; the library checks run on the shipped production table.

status	Meaning
1	Q-table load failed
2	size-distribution load failed
3	aligned scattering table load failed (only when `scatmat_path` was supplied — a host that asked for it depends on it)
4	a polarized Q table requested explicitly through `qpol_path` could not be read (an implicit default degrades gracefully instead)
5	`load_polarized_optics = .false.` combined with an explicit polarized-optics path — a contradictory request
6	astrodust dielectric function load failed (extreme-ultraviolet band only)
7	`lam_min` below the astrodust dielectric function's own shortest wavelength (extreme-ultraviolet band only)
8	the extreme-ultraviolet polarized table could not be read while the grid reaches into that band and polarized optics were asked for
9	the extreme-ultraviolet band of the grid runs outside the wavelengths the companion polarized table covers (0.0124–0.0912 μm)

Table 11: Status codes of `sed_init` and `build_astrodust` in v1.20.

Item	Size
compressed file	4.1 MB
scratch file when expanded	17.2 MB
raw arrays `qext_j`, `qabs_j`, `qsca_j`	13.7 MB, transient
derived `qpol_*`, `qran_*`	7.6 MB, kept
resident after `sed_init`	7.6 MB
time	about 1 s of the 3.1 s model build

Table 12: Cost of loading the orientation-resolved table. The orientation-resolved arrays exist only between the read and the formation of the derived combinations.